WEEK 11

에러와 디버깅

주요 학습 목표

- Thonny 디버깅
- 에러 유형 7가지
- try/except 예외 처리

에러와 디버깅

11주차 개요: 에러와 디버깅

프로그래밍 과정에서 필연적으로 발생하는 **오류(Error)**를 이해하고, **디버깅(Debugging)** 도구를 활용하여 원인을 찾아 수정하며, **예외 처리(Exception Handling)**를 통해 안정적인 프로그램을 만드는 방법을 학습합니다.

학습 목표

- 에러의 종류와 원인 파악하기
- **Thonny IDE** 디버깅 도구 활용법 익히기
- 예외 처리 구문으로 프로그램 안정성 확보

주요 도구

- **Thonny Debugger**: 라인 단위 실행
- **변수창(Variables)**: 값 변화 추적
- **중지점(Breakpoint)**: 실행 제어

실습 주제

- 7가지 주요 에러 유형 분석
- 평균 계산기 디버깅 실습
- **try-except** 구문 활용 예제

디버깅(Debugging) 정의와 역사

프로그래밍 과정에서 필연적으로 발생하는 오류를 찾아내고 수정하는 과정인 '디버깅'의 개념과 그 흥미로운 유래에 대해 알아봅니다.

핵심 포인트

1. **버그(Bug)**: 프로그램의 논리적 오류나 비정상적 연산
2. **디버깅(Debugging)**: 오류의 원인을 밝히고 수정하는 작업
3. **유래**: 1947년 컴퓨터 회로에서 발견된 실제 '나방'

디버깅이란?

컴퓨터 프로그램의 논리적인 오류(Logic Error)나 오작동을 일으키는 **버그(Bug)**를 탐지하고 제거하여 정상적으로 동작하게 만드는 과정입니다.

역사적 유래 (1947년)

초기 컴퓨터 Mark II의 회로 기판 사이에 낀 **나방(Moth)**이 오작동의 원인임을 발견하고 이를 제거(Debug)한 것에서 용어가 유래되었습니다.

```
# The First Computer Bug Log (1947)
DATE: "September 9, 1947"
TIME: "15:45"
LOG: "Relay #70 Panel F (moth) in relay."
# "First actual case of bug being found."
```

Thonny 디버깅 기능

오류 해결을 위한 강력한 도구

Thonny IDE는 초보자도 쉽게 사용할 수 있는 직관적인 디버깅 기능을 제공합니다. 라인 단위로 코드를 실행하며 변수의 변화와 프로그램의 흐름을 상세하게 추적할 수 있습니다.

- **변수창 (Variables)** — 실시간 값 확인
- **디버그 모드** — 상세 추적 시작
- **중지점 (Breakpoint)** — 특정 위치 일시정지
- **단계 들어가기 (F7)** — 한 줄씩 실행

에러 유형 Part 1: 주요 문법 에러

IndentationError

```
if True:
  print("Hello") # Correct
print("Error") # No indent
```

들여쓰기 오류입니다. 파이썬에서는 들여쓰기가 문법의 일부이므로, 블록(Block)을 구분할 때 반드시 일관된 간격(보통 공백 4칸)을 사용해야 합니다.

SyntaxError

```
print("Hello" <EOF>
# 괄호를 닫지 않음
```

문법 오류입니다. 괄호를 닫지 않거나, 콜론(:)을 빠뜨리는 등 파이썬의 문법 규칙을 어겼을 때 발생합니다. 코드가 실행되기 전에 감지됩니다.

NameError

```
print(my_score)
# 정의되지 않은 변수
```

이름 오류입니다. 변수나 함수의 이름을 오타로 잘못 입력했거나, 선언(정의)하기 전에 사용하려고 할 때 발생합니다.

Tip: 에러 메시지의 맨 마지막 줄을 보면 에러의 종류와 원인을 가장 정확하게 알 수 있습니다. 줄 번호(Line number)를 확인하세요!

에러 유형 Part 2

ValueError

부적절한 값을 입력하거나 값이 존재하지 않을 때 발생합니다. 예: 숫자가 아닌 문자열을 정수로 변환하려 할 때.

TypeError

서로 다른 데이터 타입 간에 연산이 불가능할 때 발생합니다. 예: 문자열과 숫자를 더하려고 할 때.

IndexError

리스트나 문자열 등에서 범위를 벗어난 인덱스 번호를 참조할 때 발생합니다.

AttributeError

객체에 존재하지 않는 메서드나 속성을 호출할 때 발생합니다. 예: 리스트에 없는 함수 호출.

```
print(10 + "20")   → TypeError: unsupported operand type(s) for +: 'int' and 'str'
int("abc")         → ValueError: invalid literal for int() with base 10: 'abc'
```

예외 처리와 입력 검증

try-except 구문의 활용

프로그램 실행 중 예기치 않은 에러(예: 문자를 숫자로 변환)가 발생할 때 프로그램이 비정상 종료되는 것을 막습니다. `try-except` 구문과 반복문을 결합하여, 사용자가 올바른 정수를 입력할 때까지 재입력을 요구하는 견고한 코드를 작성합니다.

try
에러 감지
(입력 시도)

while
무한 반복
(재입력 요구)

except
에러 처리
(경고 메시지)

11주차
요약 및 예고

이번 주에는 Thonny IDE를 활용하여 코드의 오류를 찾아내는 디버깅 기법과 다양한 파이썬 에러 유형을 학습했습니다. 또한, try-except 구문을 통해 예외 상황을 안전하게 처리하는 방법을 실습했습니다.

핵심 포인트 & 다음 주차:

01
디버깅 도구: Thonny & 에러 분석

02
예외 처리: try-except 구조

NEXT
AI 프로그래밍: 정보 모델링 기초